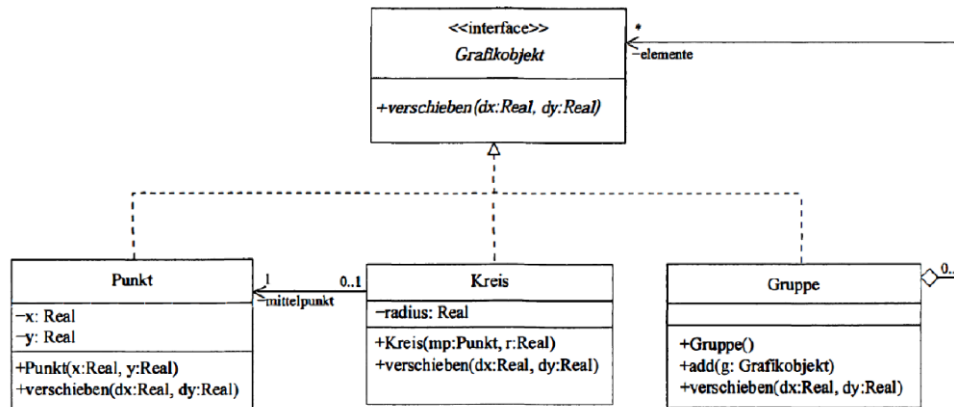


Das Entwurfsmuster Kompositum – Composite Pattern

Das Composite-Pattern wird in erster Linie dazu verwendet, um Fallunterscheidungen zu vermeiden. Dazu werden alle Gemeinsamkeiten der Klassen in einem Interface bzw. einer abstrakten Klasse zusammengefasst. Zusätzlich existiert dann für jede mögliche Ausprägung eine eigene Klasse mit eigenem Verhalten für die in der abstrakten Klasse / Interface festgelegten Methoden.



Eine **abstrakte Klasse** unterscheidet sich von einer regulären Klasse nur insoweit, als dass sie nicht Instanzieren werden kann – man kann also kein Objekt dieser Klasse bauen.

Entsprechend hat eine abstrakte Klasse auch keinen Konstruktor.

Methoden und Attribute können aber weiterhin wie gewohnt umgesetzt werden (oder explizit **abstract** deklariert werden -> Stub). In Klassen, die von der abstrakten Klasse erben, kann dann die Standardimplementierung aus der Oberklasse genutzt werden oder die Methode mittels **@Override** überschrieben werden, um die Funktionsweise zu verändern.

Dies ist von **Interfaces** abzugrenzen, da diese nur „Stubs“ – hier **Methodensignaturen** – enthalten können, keine Implementationen. Diese müssen dann in den Unterklassen implementiert werden; Das Überschreiben der Methoden aus der abstrakten Oberklasse ist hingegen optional.

Liste in perfekter Komposition

Im Fall der Liste möchten wir die Fallunterscheidungen bei der Verwaltung der Rekursionsfälle für das letzte Listenelement vermeiden. Statt also den **successor** auf null zu prüfen, wird nun die Logik aufgeteilt in zwei verschiedene Klassen: Node (Knoten) und Tail (Abschluss), die von der abstrakten Klasse ListElement erben. (vgl. S. 37)

Aufgabe 1: Erklären Sie, an welchen Stellen im Code nun Datentypen geändert werden müssen (von – zu) und warum.

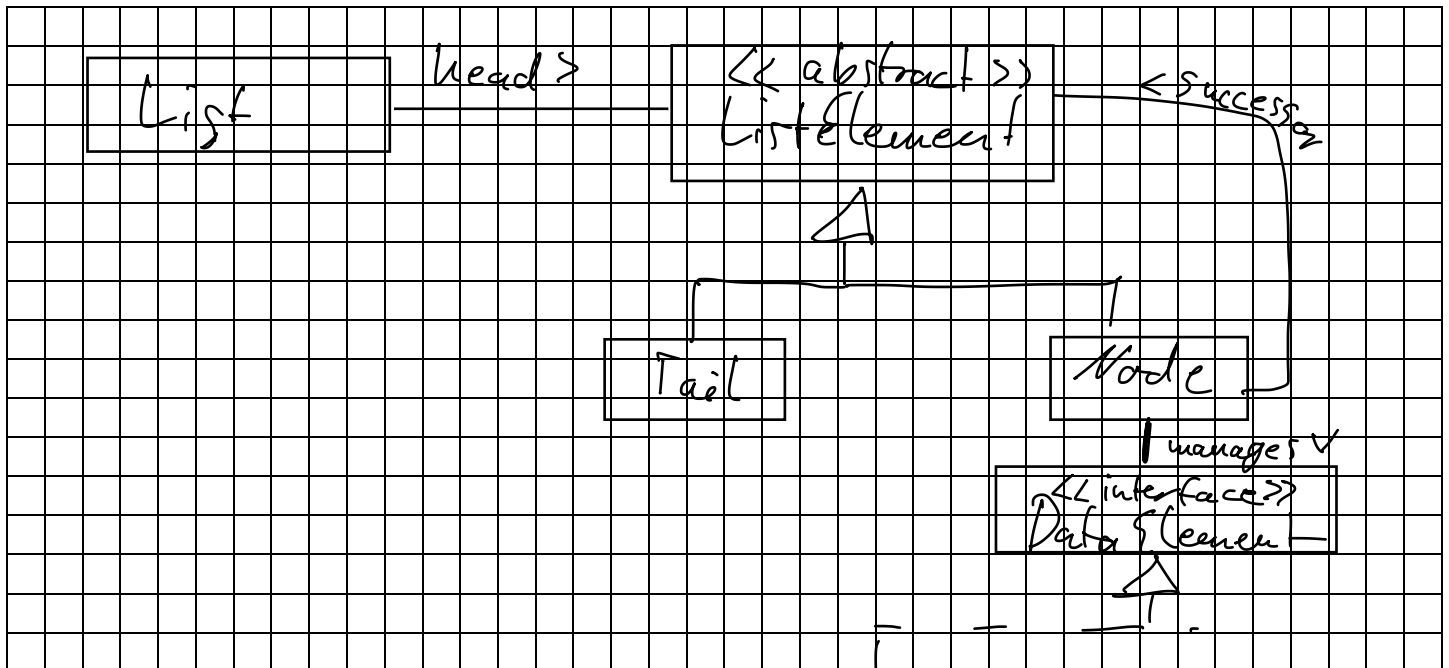
in List ist das Attribut head nun vom Typ ListElement und wird als Tail initialisiert; Dies ermöglicht es einheitlich Node und Tail Objekte anzusprechen; Tail hier als Initialisierung, da zu Beginn Liste leer;

Leere Liste: In diesem Fall referenziert das Attribut head (anfang) einfach den Abschlussknoten.

Aufgabe 2: Zeichnen Sie das Objektdiagramm einer Liste mit 2 Knoten und einer leeren Liste.

Die Tail hat selbst keinen Nachfolger und keine Daten!

Aufgabe 3: Zeichnen Sie ein vereinfachtes Klassendiagramm mit der neuen Klasse **Tail** und der abstrakten Klasse **ListElement** sowie den bestehenden Klassen für **List**, **Node**, **DataElement** und **Patient/Customer**. (vgl. S. 41)



Die Methodensignatur in **ListElement**

```
public abstract Node add(DataElement data);
```

Add Methode in der **Node** Klasse

```
@Override
public Node add(DataElement data) {
    successor = successor.add(data);
    return this;
}
```

Überschreibt Methode aus der abstrakten Oberklasse; Rekursiver Aufruf auf successor Rückgabe des passenden successors, Reihenfolge wiederherstellen bzw. successor-Attribut passend setzen

Add Methode in der **Tail** Klasse

```
@Override
public Node add(DataElement data)
{
    Node temp = new Node(data);
    temp.setSuccessor(this);
    return temp;
}
```

↓
tail

Überschreibt Methode; Neue Node wird erstellt mit data; Neue Node erhält tail als successor; Die neue Node wird an die aufrufende Node als neuer successor returned.

Aufgaben 4a) Erkunden Sie das Projekt 1-4-composite_pattern und notieren Sie weitere geänderte Methoden. Vergleichen Sie hierzu auch im Buch Seiten 38/39

ListElement remove (DC d), ListElement addBefore(...),
Node sortedInsert (...), ...

b) Verändern Sie in der Klasse **Main** die Aufrufe und beobachten Sie das Verhalten. Nutzen Sie hierzu auch den Debugger.